

Chapter 01

Introduction to LangChain

Why LangChain exists, what problem it solves, and how it fits into the architecture of LLM-powered applications.

WHAT THIS CHAPTER COVERS

- ▶ What LangChain is
- ▶ Why LangChain is needed
- ▶ Semantic search vs. keyword search
- ▶ High-level & low-level system design
- ▶ Core engineering challenges it solves
- ▶ Chains, memory & model-agnostic design
- ▶ What you can build with LangChain
- ▶ LangChain vs. LlamaIndex vs. Haystack

Introduction to LangChain

1. What is LangChain?

Definition

LangChain is an open-source framework for developing applications powered by Large Language Models (LLMs). In other words, if you want to build any application that relies on an LLM, LangChain is the framework that helps you build it faster and more reliably.

Why It Exists

Any single-sentence definition of LangChain undersells its importance. The real value of the framework only becomes clear once you understand *why* it was needed in the first place. Before studying what a technology is, it is far more useful to understand what problem made that technology necessary. LangChain was created because building a production-grade LLM application involves far more than just calling an LLM — it involves loading documents, splitting text, generating embeddings, storing and querying vectors, retrieving relevant context, and finally passing everything to an LLM in the right format. Wiring all of this together by hand is slow, repetitive, and fragile.

Problem It Solves

LangChain solves the problem of **orchestration** — connecting many independent components (document loaders, text splitters, embedding models, vector databases, and LLMs) into a single, coherent pipeline, without forcing the developer to write large amounts of boilerplate integration code for each component.

IMPORTANT

LangChain does not replace the LLM. The LLM still performs the actual reasoning and text generation. LangChain's job is to orchestrate everything *around* the LLM so that the right context reaches it in the right format.

Practical Use Cases (Preview)

LangChain is commonly used to build conversational chatbots, AI knowledge assistants that are grounded in private data, autonomous AI agents that can take actions, workflow automation tools, and summarization / research assistants. These are covered in detail later in this chapter.

INTERVIEW TIP

If asked “What is LangChain?” in an interview, do not stop at the definition. Follow it with the orchestration problem it solves — this shows you understand the framework at a systems level, not just as a buzzword.

COMMON MISTAKE

Treating LangChain as “an LLM” or as a replacement for a model provider. LangChain is a coordination layer; it still needs an underlying LLM (via API or self-hosted) to actually understand language and generate answers.

BEST PRACTICE

Before reaching for LangChain, be able to explain the pipeline you would need to hand-code without it. This makes the value of every LangChain abstraction concrete rather than abstract.

🔄 Quick Revision — What is LangChain?

- LangChain = open-source framework for building LLM-powered applications.
- Its core value is **orchestration**, not intelligence — the LLM still does the reasoning.
- It exists because wiring loaders, splitters, embeddings, vector stores, and LLMs by hand does not scale.
- Understand the problem (motivating case study) before memorizing the definition.

2. The Motivating Case Study: “Chat With Your PDF”

The Original Idea

A useful way to understand why LangChain matters is to walk through a real product idea: an application where a user uploads a PDF (for example, a machine learning textbook) and can both *read* it and *chat* with it. Through the chat interface, the user could ask the application to simplify a page for a five-year-old, generate true/false questions on a topic covered in the book, or produce revision notes for a specific concept — all grounded in the content of that specific document.

High-Level System Design

At a high level, the system needs to do the following whenever a user asks a question about their uploaded document:

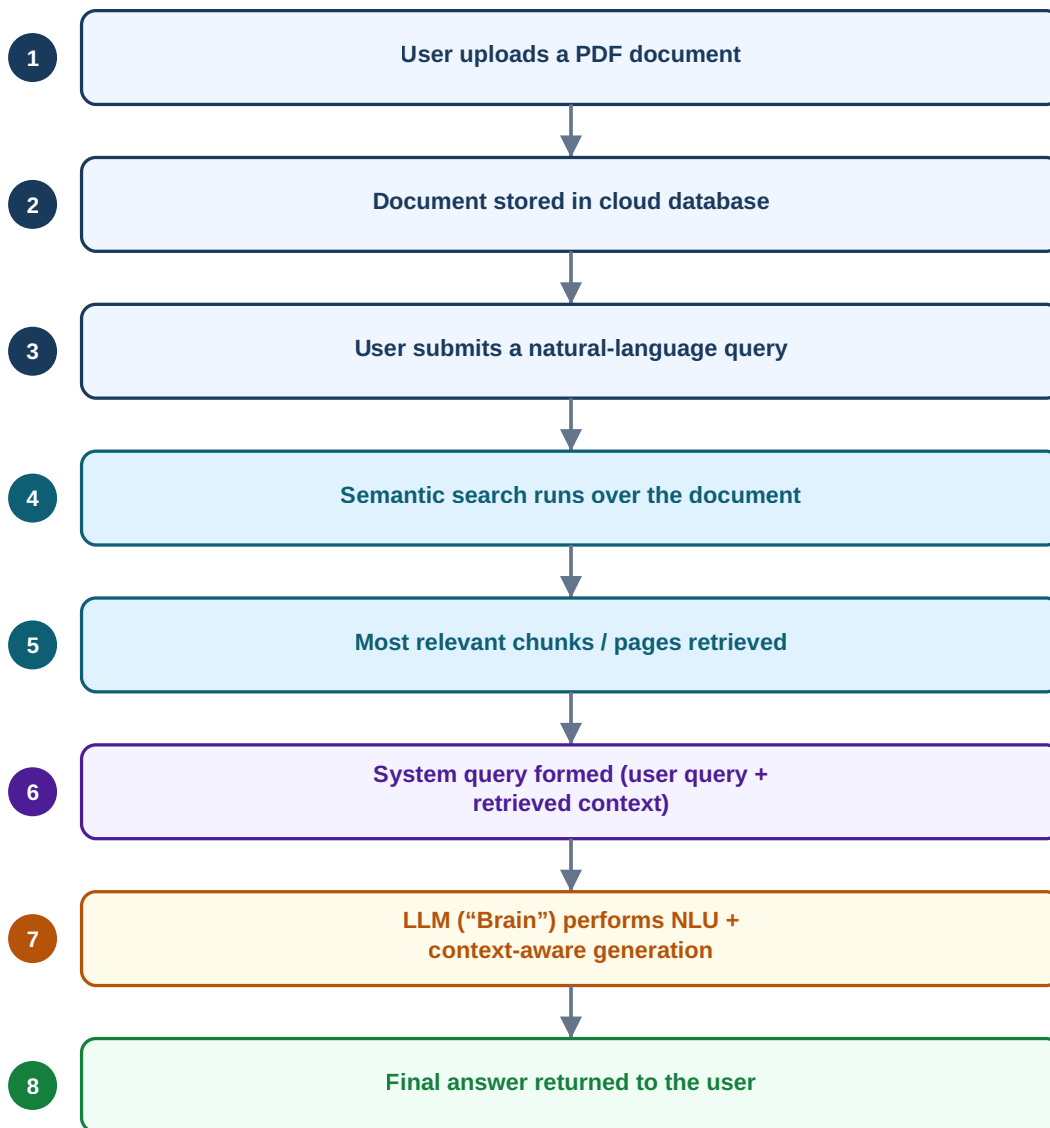


Figure 1.1 — High-level flow for a "chat with your document" application.

Why Not Just Search for Keywords?

The first, most obvious approach to "finding where a topic is discussed" is a **keyword search**: scan the document for the exact words in the query. This is inefficient. A query like "what are the assumptions of linear regression" may match the word "assumptions" on dozens of unrelated pages, producing a large, noisy result set. A **semantic search** instead searches by *meaning*, returning far fewer but far more relevant pages. This is discussed in detail in the next section.

Why Not Just Send the Entire Document to the LLM?

A natural follow-up question is: if the LLM can understand a query and extract an answer from given pages, why not skip the search step entirely and send the LLM the *whole* book? The answer is the same reason a student pointing a teacher to a specific page number gets a faster, more accurate answer than a student who simply hands over the entire textbook. Sending the full document for every query is computationally expensive and tends to produce less focused answers. Retrieving only the most relevant chunks first, and sending *only those* to the LLM, is both cheaper and more accurate.

REAL WORLD INSIGHT

This same PDF-chat idea, first considered around 2014–2015 (before the current wave of generative AI), is now essentially trivial to build because both the retrieval problem (semantic search) and the reasoning problem (LLMs) have mature, off-the-shelf solutions today.

🔄 Quick Revision — Motivating Case Study

- Goal: let a user both read and chat with an uploaded document.
- Naive keyword search over-retrieves and is inefficient; semantic search retrieves by meaning.
- Never send the entire document to the LLM on every query — retrieve only relevant chunks first.
- The retrieved chunks + the user's query are combined into a single 'system query' sent to the LLM.

3. Understanding Semantic Search

Definition

Semantic search is a retrieval technique that finds content based on *meaning* rather than exact word overlap. It is the mechanism that lets the system decide which pages of a document are actually relevant to a user's question.

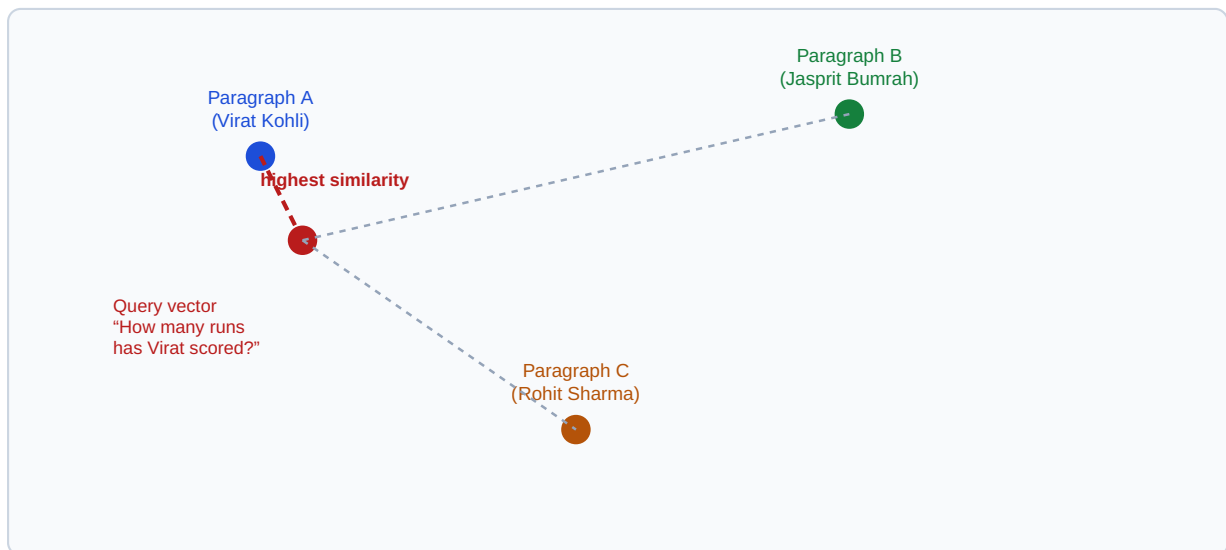
Keyword Search vs. Semantic Search

Aspect	Keyword Search	Semantic Search
Matching basis	Exact word / substring match	Meaning of the query
Typical result set	Large, often noisy	Small, highly relevant
Underlying representation	Raw text	Vector embeddings
Handles synonyms / paraphrasing	No	Yes
Computational cost per query	Low	Moderate (embedding + similarity search)

How It Works — Step by Step

1. Every chunk of text (a paragraph, page, or section) is converted into an **embedding** — a vector, i.e. a fixed-length list of numbers that represents its semantic meaning. Techniques used to generate embeddings include Word2Vec, Doc2Vec, and transformer-based embeddings such as those produced by BERT-style models.
2. All chunk embeddings are stored (for example, as vectors in an N -dimensional space).
3. When a user submits a query, the query itself is converted into an embedding using the *same* embedding model, so it lands in the same vector space as the chunks.

4. The similarity between the query vector and every chunk vector is computed (for example, using cosine similarity or distance).
5. The chunks whose vectors are closest to the query vector are returned as the most relevant results.



Each paragraph and the query are embedded into the same N-dimensional vector space; the closest vector wins.

Figure 1.2 — Example: three paragraphs (about three cricketers) and a query, all embedded into the same vector space. The closest vector wins.

Example

Given three paragraphs — one about Virat Kohli, one about Jasprit Bumrah, and one about Rohit Sharma — and the query “How many runs has Virat scored?”, semantic search embeds all three paragraphs and the query into the same vector space, computes similarity between the query vector and each paragraph vector, and returns the Virat Kohli paragraph because its vector is closest to the query vector.

COMMON MISTAKE

Using a different embedding model for the query than the one used for the stored document chunks. Query and document embeddings must come from the same embedding model, otherwise their vector spaces are not comparable.

Quick Revision — Semantic Search

- Embedding = converting text into a vector (a set of numbers) that captures its meaning.
- Query and documents must be embedded using the same embedding model.
- Relevance is determined by vector similarity/distance, not exact word match.
- Semantic search returns fewer, more meaningful results compared to keyword search.

4. Detailed (Low-Level) System Design

With semantic search understood, the full pipeline for the document-chat system can now be described precisely, split into two phases: an **ingestion pipeline** that runs once per uploaded document, and a **query-time pipeline** that runs on every question the user asks.

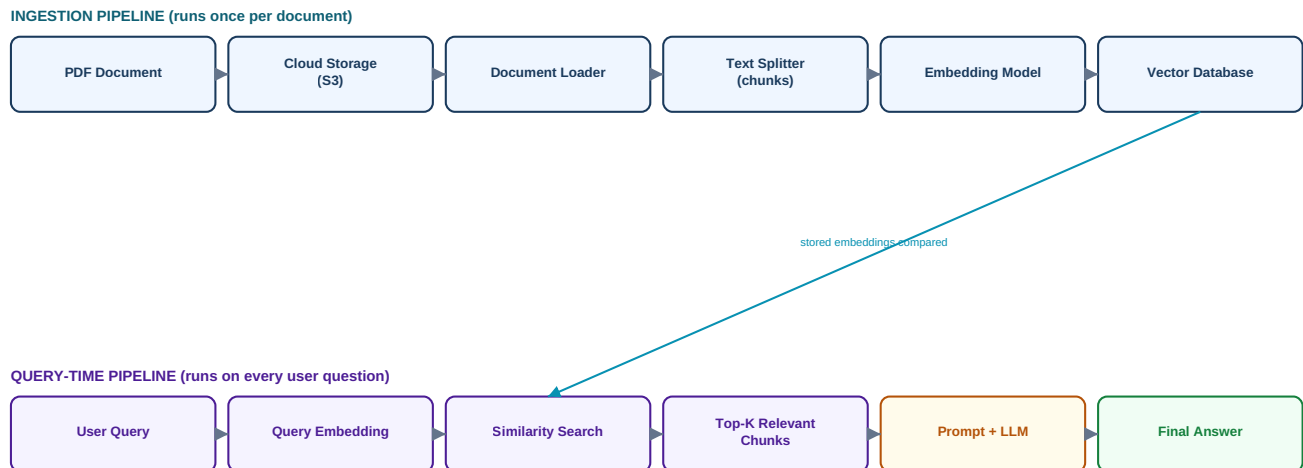


Figure 1.3 — Full ingestion and query-time pipeline for the document-chat system.

Ingestion Pipeline (Step by Step)

1. **Upload & storage** — the uploaded PDF is stored in cloud storage (for example, AWS S3).
2. **Document loading** — a document loader brings the PDF's content into the application.
3. **Text splitting** — the document is broken into smaller chunks. The split can be done by chapter, by page, or even by paragraph, depending on the use case.
4. **Embedding generation** — each chunk is passed through an embedding model, producing one vector per chunk.
5. **Vector storage** — all chunk embeddings are stored in a (vector) database so they can be queried later.

Query-Time Pipeline (Step by Step)

1. The user's text query is embedded using the same embedding model used during ingestion.
2. The query vector is compared against every stored chunk vector, and the closest matches (for example, the top five) are retrieved.
3. The retrieved chunks and the original user query are combined into a single **system query**.

```
# Conceptual shape of the system query sent to the LLM
system_query = f"""
Context: {{retrieved_chunks}}
User Question: {{user_query}}
Instruction: Answer the question using only the context above.
"""
```

1. The system query is sent to the LLM (referred to informally as the application's “brain”), which needs two capabilities:
 - **Natural Language Understanding (NLU)** — to correctly interpret the user's query, regardless of the language it is written in.
 - **Context-aware text generation** — to read the retrieved chunks and produce an answer grounded in them.
2. The LLM's generated response is returned to the user as the final answer.

BEST PRACTICE

Keep the ingestion pipeline (expensive, run once) and the query-time pipeline (cheap, run repeatedly) conceptually separate. This distinction directly maps to how vector databases and caching strategies are designed in real systems.

Quick Revision — System Design

- Ingestion pipeline: Load → Split → Embed → Store (runs once per document).
- Query pipeline: Embed query → Similarity search → Retrieve top-K chunks → Build system query → LLM → Answer.
- The LLM needs both NLU (to understand the query) and context-aware generation (to answer from retrieved chunks).
- This entire pipeline, hand-coded, has many moving parts and many failure points — the exact gap LangChain fills.

5. Three Core Engineering Challenges

Building the system above surfaces three distinct engineering challenges. Understanding how each one is solved today explains exactly where LangChain fits.

#	Challenge	Description	How It Is Solved Today
1	Understanding & generating language	The system's “brain” must understand a query (NLU) and generate a context-aware answer from retrieved text.	Solved by modern LLMs , made possible by the 2017 Transformer paper and the subsequent BERT / GPT line of research.
2	Hosting & compute	LLMs are extremely large deep learning models; self-hosting and serving them requires heavy infrastructure engineering and is costly.	Solved by LLM APIs from providers such as OpenAI and Anthropic — you call an API and pay only for what you use, instead of hosting the model yourself.
3	Orchestration	The system has many moving components (storage, loader, splitter, embedding model, vector database, LLM) and many tasks that must be chained together correctly.	Solved by LangChain , which provides built-in, plug-and-play integrations so components can be connected without writing custom boilerplate for each one.

INTERVIEW TIP

A strong answer to “why do we need LangChain if LLMs are already so powerful?” is to separate the three challenges above: LLMs solve understanding/generation, LLM APIs solve hosting/compute, and LangChain solves orchestration. LangChain does not compete with LLMs — it sits around them.

🔄 Quick Revision — Core Challenges

- Challenge 1 (NLU + generation) → solved by LLMs.
- Challenge 2 (hosting/compute) → solved by LLM provider APIs (pay-per-use).
- Challenge 3 (orchestration of many components/tasks) → solved by LangChain.
- If every provider swap (e.g., OpenAI → Google) required hand-editing a large hard-coded pipeline, that itself is a sign orchestration tooling is missing.

6. Benefits of LangChain

1. CHAINS

LangChain gets its name from the concept of **chains**. A chain lets you connect different components and tasks into a pipeline-like structure, where the output of one component automatically becomes the input of the next — without manually writing that wiring code. Chains can be simple (sequential), or more advanced: **parallel chains** (multiple tasks run at once) and **conditional chains** (branching logic), giving an expressive way to build pipelines of arbitrary complexity.

2. MODEL-AGNOSTIC DEVELOPMENT

LangChain is designed so that the underlying model (or embedding model, or vector store) can be swapped with minimal code changes. Switching from one LLM provider to another, in principle, can be a change of a couple of lines rather than

a rewrite of your business logic. This lets developers focus on core application logic instead of being tightly coupled to one vendor.

3. A COMPLETE ECOSYSTEM

LangChain provides a wide catalogue of integrations for every component type: many document loaders (cloud storage, local files, PDFs, and more), many text splitters, many embedding models, and many vector databases. Whatever component a project needs, LangChain is likely to already have an interface for it.

4. MEMORY & STATE HANDLING

LangChain provides conversation memory, so a system can retain context across turns. For example, if a user asks for the assumptions of linear regression and then asks “also give me a few interview questions on this algorithm” without repeating “linear regression,” a system with conversational memory can still resolve *this* correctly.

Benefit	What It Gives You
Chains	Declarative pipelines; automatic input/output wiring between components; sequential, parallel, and conditional composition.
Model-agnostic design	Swap LLMs, embedding models, or vector stores with minimal code changes.
Complete ecosystem	Ready-made loaders, splitters, embedding models, and vector database integrations.
Memory & state	Conversation memory so follow-up questions resolve correctly without repeated context.

COMMON MISTAKE

Forgetting to add conversational memory in multi-turn assistants, then being surprised that follow-up questions (‘what about the second one?’) fail because the system has no memory of the previous turn.

Quick Revision — Benefits

- Chains: pipe components together; output of one = input of next, automatically.
- Model-agnostic: change providers/components without rewriting core logic.
- Ecosystem: loaders, splitters, embedding models, and vector DBs are all pre-integrated.
- Memory: enables context-aware, multi-turn conversations.

7. What Can You Build With LangChain?

LangChain is used across a wide range of LLM-powered products. The most common categories are summarized below.

Use Case	Description	Example
Conversational chatbots	Handle the first layer of customer communication at scale, escalating to a human only when needed.	Customer support for internet-first companies.
AI knowledge assistants	Like a chatbot, but grounded in a specific knowledge base or dataset.	A course platform chatbot that can answer doubts about the specific lecture being watched.
AI agents	Go beyond conversation — they can take actions and use tools to complete tasks.	An agent that books the cheapest flight ticket between two cities on a given date.
Workflow automation	Automate multi-step workflows at a personal, professional, or company level.	Automating a repetitive multi-step business process.
Summarization & research helpers	Process documents too large for a standard chat context window, including private company data.	An internal "chat with our documentation" tool.

REAL WORLD INSIGHT

AI agents are considered one of the most significant emerging categories of LLM applications — systems that do not just talk, but can also execute tasks on the user's behalf using tools.

Quick Revision — Use Cases

- Chatbots: first line of customer communication at scale.
- Knowledge assistants: chatbots grounded in a specific dataset.
- Agents: chatbots that can also take actions using tools.
- Workflow automation and summarization/research tools round out the common use cases.

8. LangChain and Its Alternatives

LangChain is not the only framework for building LLM-powered applications. Two other well-known frameworks are **LlamaIndex** and **Haystack**.

Framework	Positioning	Notes
LangChain	General-purpose LLM application framework with strong chain/orchestration abstractions.	Broad ecosystem; widely adopted across many types of LLM applications.
LlamaIndex	Framework popular for building LLM applications, especially retrieval-centric ones.	Considered slightly more popular for certain workflows; has significant community adoption.
Haystack	A similar type of LLM application framework/library.	Also allows building LLM-powered applications with relative ease.

The right choice between these depends on factors such as pricing, ecosystem fit, and which tool feels best suited to the specific product being built. A detailed pros-and-cons comparison is only meaningful once each framework has been studied individually.

Quick Revision — Alternatives

- LangChain is not the only option — LlamaIndex and Haystack are notable alternatives.
- Framework choice depends on pricing, ecosystem, and project fit rather than a single universal answer.

K. Key Takeaways

- LangChain is an open-source framework for building LLM-powered applications — its core value is [simplicity](#), not intelligence.
- It exists because assembling document loaders, text splitters, embedding models, vector databases, and LLMs by hand is slow, repetitive, and fragile.
- Semantic search (meaning-based retrieval using vector embeddings) is far more efficient than keyword search, and is the retrieval mechanism behind most LLM applications that work over private documents.
- A production LLM application has an [ingestion pipeline](#) (load, split, embed, store — runs once) and a [query pipeline](#) (embed query, retrieve, build system query, generate answer — runs every time).
- Three core challenges appear when building such systems: language understanding/generation (solved by LLMs), hosting/compute (solved by LLM APIs), and orchestration (solved by LangChain).
- LangChain's key benefits are chains, model-agnostic development, a complete ecosystem of integrations, and built-in memory/state handling.
- Common LangChain use cases include chatbots, knowledge assistants, AI agents, workflow automation, and summarization/research tools.
- LangChain is one of several frameworks in this space; LlamaIndex and Haystack are notable alternatives.

R. Revision Sheet

LangChain, in one line

Open-source framework that orchestrates the components needed to build LLM-powered applications.

Semantic vs. keyword search

Semantic search matches by meaning using vector embeddings; keyword search matches exact words and over-retrieves.

Ingestion pipeline

Load → Split into chunks → Embed → Store in a vector database. Runs once per document.

Query-time pipeline

Embed query → Similarity search → Retrieve top-K chunks → Build system query → LLM → Answer.

Three challenges LangChain-adjacent tools solve

Understanding/generation → LLMs. Hosting/compute → LLM APIs. Orchestration → LangChain.

Core LangChain benefits

Chains, model-agnostic design, a complete component ecosystem, and conversation memory.

What you can build

Chatbots, knowledge assistants, AI agents, workflow automation, summarization/research tools.

Alternatives

LlamaIndex and Haystack are the two most commonly cited alternatives to LangChain.

I. Interview Questions

- Q1.** What problem does LangChain solve that an LLM alone does not solve?
- Q2.** Explain the difference between keyword search and semantic search.
- Q3.** Walk through the ingestion pipeline and the query-time pipeline for a document-based question-answering system.
- Q4.** Why is it inefficient to send an entire document to an LLM on every query instead of retrieving relevant chunks first?
- Q5.** What are embeddings, and why must the query and the stored documents be embedded using the same model?
- Q6.** What are the three core engineering challenges in building an LLM-powered application, and how is each one typically solved today?
- Q7.** What is a "chain" in LangChain, and why is it central to the framework's design?
- Q8.** What does it mean for LangChain to be "model-agnostic," and why does this matter for a production system?
- Q9.** How does LangChain's memory/state handling help with multi-turn conversations?
- Q10.** Name two alternatives to LangChain and describe how you would decide between them.

F. Further Reading

- LangChain official documentation — python.langchain.com
- LlamaIndex official documentation — for a retrieval-centric alternative framework
- Haystack official documentation — for another LLM application framework
- "Attention Is All You Need" (2017) — the Transformer paper that underpins modern LLMs
- Foundational papers on BERT and GPT — for background on the NLU and generation capabilities LLMs provide